

Transformers: From Math to Implementation

From Linear Algebra to Self-Attention

A hands-on guide to understanding Transformers from the ground up.

Every concept is explained with math, then implemented in PyTorch with executable code.

Table of Contents

- Chapter 1: Math Foundations for Machine Learning
 - Chapter 2: The Simplest Neural Network
 - Chapter 3: Loss Functions, Softmax, and Temperature
 - Chapter 4: Math Behind the Transformer
 - Chapter 5: The Complete Transformer
-

Chapter 1: Math Foundations for Machine Learning

1.1 Scalars, Vectors, Matrices, and Tensors

Machine learning operates on multi-dimensional arrays of numbers. Let's build intuition from the ground up.

- Scalar: A single number, e.g. $x = 3.14$
- Vector: A 1-D array, e.g. $v = [1, 2, 3]$
- Matrix: A 2-D array with rows and columns
- Tensor: An n-dimensional generalization (what PyTorch is named after)

```
import torch

scalar = torch.tensor(3.14)
vector = torch.tensor([1.0, 2.0, 3.0])
matrix = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
tensor_3d = torch.randn(2, 3, 4)

print(f"Scalar: shape={scalar.shape}, value={scalar.item():.2f}")
print(f"Vector: shape={vector.shape}, values={vector.tolist()}")
print(f"Matrix: shape={matrix.shape}")
print(f"3D Tensor: shape={tensor_3d.shape}")
```

```
Scalar: shape=torch.Size([]), value=3.14
Vector: shape=torch.Size([3]), values=[1.0, 2.0, 3.0]
Matrix: shape=torch.Size([2, 2])
3D Tensor: shape=torch.Size([2, 3, 4])
```

1.2 Dot Product and Matrix Multiplication

The dot product of two vectors a and b is:

$$a \cdot b = \sum(a_i * b_i) \text{ for } i = 1..n$$

Matrix multiplication $C = A @ B$ where $C_{ij} = \sum(A_{ik} * B_{kj})$:

- A is $(m \times n)$, B is $(n \times p)$, result C is $(m \times p)$
- The inner dimensions must match

```
import torch

a = torch.tensor([1.0, 2.0, 3.0])
b = torch.tensor([4.0, 5.0, 6.0])
dot = torch.dot(a, b)
print(f"Dot product: {a.tolist()} . {b.tolist()} = {dot.item()}")
print(f"   = 1*4 + 2*5 + 3*6 = {1*4 + 2*5 + 3*6}")

A = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
B = torch.tensor([[5.0, 6.0], [7.0, 8.0]])
C = A @ B
print(f"\nMatrix multiply A @ B:")
print(f"   A = {A.tolist()}")
print(f"   B = {B.tolist()}")
print(f"   C = {C.tolist()}")
```

```
Dot product: [1.0, 2.0, 3.0] . [4.0, 5.0, 6.0] = 32.0
             = 1*4 + 2*5 + 3*6 = 32
```

```
Matrix multiply A @ B:
A = [[1.0, 2.0], [3.0, 4.0]]
B = [[5.0, 6.0], [7.0, 8.0]]
C = [[19.0, 22.0], [43.0, 50.0]]
```

1.3 Norms

Norms measure the "size" of a vector:

- L1 norm: $\|x\|_1 = \sum(|x_i|)$ — Manhattan distance
- L2 norm: $\|x\|_2 = \sqrt{\sum(x_i^2)}$ — Euclidean distance
- L-inf norm: $\|x\|_{\infty} = \max(|x_i|)$

```
import torch

x = torch.tensor([3.0, -4.0])
print(f"x = {x.tolist()}")
print(f"L1 norm  = |3| + |-4| = {torch.norm(x, p=1).item()}")
print(f"L2 norm  = sqrt(9 + 16) = {torch.norm(x, p=2).item()}")
print(f"Linf norm = max(3, 4) = {torch.norm(x, p=float('inf')).item()}")
```

```
x = [3.0, -4.0]
L1 norm  = |3| + |-4| = 7.0
L2 norm  = sqrt(9 + 16) = 5.0
Linf norm = max(3, 4) = 4.0
```

1.4 Derivatives and Gradients

The derivative $f'(x)$ tells us how f changes when x changes by a tiny amount.

For $f(x) = x^2$: $f'(x) = 2x$

The gradient generalizes derivatives to multiple variables:

- For $f(x, y) = x^2 + 3xy$, the gradient is $[2x + 3y, 3x]$

PyTorch computes gradients automatically with autograd:

```
import torch

x = torch.tensor(3.0, requires_grad=True)
f = x ** 2
f.backward()
print(f"f(x) = x^2, at x=3: f={f.item()}, df/dx={x.grad.item()}")
print(f"    (analytical: 2*3 = 6)")

x = torch.tensor(2.0, requires_grad=True)
y = torch.tensor(5.0, requires_grad=True)
f = x**2 + 3*x*y
f.backward()
print(f"\nf(x,y) = x^2 + 3xy, at x=2, y=5:")
print(f"    df/dx = 2x + 3y = {x.grad.item()} (expected {2*2 + 3*5})")
print(f"    df/dy = 3x = {y.grad.item()} (expected {3*2})")
```

```
f(x) = x^2, at x=3: f=9.0, df/dx=6.0
    (analytical: 2*3 = 6)

f(x,y) = x^2 + 3xy, at x=2, y=5:
    df/dx = 2x + 3y = 19.0 (expected 19)
    df/dy = 3x = 6.0 (expected 6)
```

1.5 The Chain Rule

If $y = f(g(x))$, then $dy/dx = f'(g(x)) * g'(x)$.

This is the backbone of backpropagation. PyTorch chains gradients automatically.

```
import torch

x = torch.tensor(2.0, requires_grad=True)
# y = sin(x^2)
# dy/dx = cos(x^2) * 2x
y = torch.sin(x ** 2)
y.backward()
manual = torch.cos(torch.tensor(4.0)) * 4.0
print(f"y = sin(x^2), x=2")
print(f"    autograd dy/dx = {x.grad.item():.6f}")
print(f"    manual    dy/dx = cos(4)*4 = {manual.item():.6f}")
```

```
y = sin(x^2), x=2
autograd dy/dx = -2.614574
manual    dy/dx = cos(4)*4 = -2.614574
```

1.6 Probability Basics

- Probability distribution: $p(x) \geq 0$, sum of $p(x) = 1$
- Expected value: $E[x] = \sum(x_i * p(x_i))$
- Variance: $Var(x) = E[(x - E[x])^2]$

```
import torch

# Fair die
probs = torch.ones(6) / 6
values = torch.arange(1, 7, dtype=torch.float)

expected = (values * probs).sum()
variance = ((values - expected)**2 * probs).sum()
print(f"Fair die:")
print(f"  E[X] = {expected.item():.4f}")
print(f"  Var(X) = {variance.item():.4f}")
print(f"  Std(X) = {variance.sqrt().item():.4f}")
```

```
Fair die:
  E[X] = 3.5000
  Var(X) = 2.9167
  Std(X) = 1.7078
```

1.7 Information Theory: Entropy

Entropy $H(p)$ measures uncertainty in a distribution:

$$H(p) = -\sum(p_i * \log(p_i))$$

A uniform distribution has maximum entropy (most uncertain).

A one-hot distribution has zero entropy (completely certain).

```
import torch

def entropy(p):
    p = p[p > 0]
    return -(p * p.log()).sum()

uniform = torch.ones(4) / 4
peaked = torch.tensor([0.97, 0.01, 0.01, 0.01])
one_hot = torch.tensor([1.0, 0.0, 0.0, 0.0])

print(f"Uniform [0.25, 0.25, 0.25, 0.25]: H = {entropy(uniform):.4f}")
print(f"Peaked [0.97, 0.01, 0.01, 0.01]: H = {entropy(peaked):.4f}")
print(f"One-hot [1.00, 0.00, 0.00, 0.00]: H = {entropy(one_hot):.4f}")
print(f"Max possible (ln 4) = {torch.tensor(4.0).log():.4f}")
```

```
Uniform [0.25, 0.25, 0.25, 0.25]: H = 1.3863
Peaked [0.97, 0.01, 0.01, 0.01]: H = 0.1677
One-hot [1.00, 0.00, 0.00, 0.00]: H = -0.0000
Max possible (ln 4) = 1.3863
```

Chapter 2: The Simplest Neural Network

2.1 A Single Neuron (Perceptron)

A neuron computes:

$$\text{output} = \text{activation}(w \cdot x + b)$$

where w is a weight vector, x is input, b is bias, and activation is a nonlinear function.

```
import torch

# Single neuron: 3 inputs -> 1 output
torch.manual_seed(42)
x = torch.tensor([1.0, 2.0, 3.0])
w = torch.randn(3)
b = torch.randn(1)

z = torch.dot(w, x) + b          # linear combination
a = torch.sigmoid(z)             # activation

print(f"Input x      = {x.tolist()}")
print(f"Weights w    = {[f'{v:.4f}' for v in w.tolist()]}")
print(f"Bias b       = {b.item():.4f}")
print(f"z = w.x+b     = {z.item():.4f}")
print(f"sigmoid(z)    = {a.item():.4f}")
```

```
Input x      = [1.0, 2.0, 3.0]
Weights w    = ['0.3367', '0.1288', '0.2345']
Bias b       = 0.2303
z = w.x+b     = 1.5280
sigmoid(z)    = 0.8217
```

2.2 Activation Functions

Activation functions introduce nonlinearity. Without them, stacking layers would just be another linear transformation.

Function	Formula	Range
Sigmoid	$1/(1+e^{-x})$	(0, 1)
Tanh	$(e^x - e^{-x})/(e^x + e^{-x})$	(-1, 1)
ReLU	$\max(0, x)$	[0, inf)
GELU	$x * \Phi(x)$	(-0.17, inf)

```
import torch
import torch.nn.functional as F

x = torch.tensor([-2.0, -1.0, 0.0, 1.0, 2.0])
print(f"x          = {x.tolist()}")
print(f"sigmoid     = {torch.sigmoid(x).tolist()}")
print(f"tanh         = {torch.tanh(x).tolist()}")
print(f"relu          = {F.relu(x).tolist()}")
print(f"gelu          = {F.gelu(x).tolist()}")
```

```

x      = [-2.0, -1.0, 0.0, 1.0, 2.0]
sigmoid = [0.11920291930437088, 0.2689414322376251, 0.5, 0.7310585975646973, 0.8807970285415649]
tanh    = [-0.9640275835990906, -0.7615941762924194, 0.0, 0.7615941762924194, 0.9640275835990906]
relu     = [0.0, 0.0, 0.0, 1.0, 2.0]
gelu     = [-0.04550027847290039, -0.15865525603294373, 0.0, 0.8413447141647339, 1.9544997215270996]

```

2.3 A Two-Layer Network from Scratch

Let's build a 2-layer network for XOR (the classic non-linearly-separable problem):

Input (2) -> Hidden (4, ReLU) -> Output (1, Sigmoid)

```

import torch
import torch.nn as nn

# XOR data
X = torch.tensor([[0,0],[0,1],[1,0],[1,1]], dtype=torch.float32)
Y = torch.tensor([[0],[1],[1],[0]], dtype=torch.float32)

class XORNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden = nn.Linear(2, 4)
        self.output = nn.Linear(4, 1)

    def forward(self, x):
        x = torch.relu(self.hidden(x))
        x = torch.sigmoid(self.output(x))
        return x

torch.manual_seed(42)
model = XORNet()
optimizer = torch.optim.SGD(model.parameters(), lr=1.0)
loss_fn = nn.BCELoss()

for epoch in range(1001):
    pred = model(X)
    loss = loss_fn(pred, Y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    if epoch % 250 == 0:
        print(f"Epoch {epoch:4d} | Loss: {loss.item():.6f}")

print(f"\nFinal predictions:")
with torch.no_grad():
    for i in range(4):
        p = model(X[i]).item()
        print(f"  {X[i].tolist()} -> {p:.4f} (target: {Y[i].item():.0f})")

```

```

Epoch   0 | Loss: 0.806469
Epoch 250 | Loss: 0.006255
Epoch 500 | Loss: 0.002137
Epoch 750 | Loss: 0.001217
Epoch 1000 | Loss: 0.000833

```

```

Final predictions:
[0.0, 0.0] -> 0.0008 (target: 0)
[0.0, 1.0] -> 0.9995 (target: 1)
[1.0, 0.0] -> 0.9988 (target: 1)
[1.0, 1.0] -> 0.0008 (target: 0)

```

2.4 Backpropagation — How Neural Networks Learn

Backpropagation uses the chain rule to compute the gradient of the loss with respect to every parameter in the network.

Forward pass: compute the output

Backward pass: compute gradients layer by layer, from output back to input

For a simple network with one hidden layer:

$$1. z_1 = W_1 @ x + b_1$$

$$2. a_1 = \sigma(z_1)$$

Backward:

- $dL/dy_{\text{hat}} = -(y/y_{\text{hat}} - (1-y)/(1-y_{\text{hat}}))$
- $dL/dz_2 = y_{\text{hat}} - y$ (for sigmoid + BCE)
- $dL/dW_2 = dL/dz_2 @ a_1^T$
- $dL/da_1 = W_2^T @ dL/dz_2$
- $dL/dz_1 = dL/da_1 * (z_1 > 0)$ (ReLU derivative)
- $dL/dW_1 = dL/dz_1 @ x^T$

```
import torch

# Manual backpropagation for a tiny network
torch.manual_seed(0)
x = torch.tensor([[1.0, 0.5]]) # (1, 2)
y = torch.tensor([[1.0]]) # target

W1 = torch.randn(2, 3, requires_grad=True) # (2, 3)
b1 = torch.zeros(1, 3, requires_grad=True) # (1, 3)
W2 = torch.randn(3, 1, requires_grad=True) # (3, 1)
b2 = torch.zeros(1, 1, requires_grad=True) # (1, 1)

# Forward
z1 = x @ W1 + b1
a1 = torch.relu(z1)
z2 = a1 @ W2 + b2
y_hat = torch.sigmoid(z2)
loss = -(y * torch.log(y_hat) + (1-y) * torch.log(1-y_hat))

# Autograd backward
loss.backward()
print("=== Autograd gradients ===")
print(f"dL/dW1 = {W1.grad.flatten().tolist()}")
print(f"dL/dW2 = {W2.grad.flatten().tolist()}")

# Manual backward
with torch.no_grad():
    W1m = W1.detach().clone()
    b1m = b1.detach().clone()
    W2m = W2.detach().clone()
    b2m = b2.detach().clone()

    z1m = x @ W1m + b1m
    a1m = torch.relu(z1m)
    z2m = a1m @ W2m + b2m
    y_hat_m = torch.sigmoid(z2m)

    dz2 = y_hat_m - y # (1,1)
```

```

dW2 = a1m.T @ dz2                # (3,1)
da1 = dz2 @ W2m.T                # (1,3)
dz1 = da1 * (z1m > 0).float()    # ReLU grad
dW1 = x.T @ dz1                  # (2,3)

print("\n=== Manual gradients ===")
print(f"dL/dW1 = {dW1.flatten().tolist()}")
print(f"dL/dW2 = {dW2.flatten().tolist()}")
print("\nMatch!" if torch.allclose(W1.grad, dW1) else "Mismatch!")

```

```

=== Autograd gradients ===
dL/dW1 = [-0.13061867654323578, 0.0, 0.0, -0.06530933827161789, 0.0, 0.0]
dL/dW2 = [-0.5910713076591492, 0.0, 0.0]

=== Manual gradients ===
dL/dW1 = [-0.13061867654323578, 0.0, 0.0, -0.06530933827161789, 0.0, 0.0]
dL/dW2 = [-0.5910713076591492, 0.0, 0.0]

Match!

```

2.5 Gradient Descent Variants

Algorithm	Update Rule	Key Idea
SGD	$w \leftarrow lr * grad$	Simple, but noisy
Momentum	$v = \beta v + grad$; $w \leftarrow lr v$	Smooths updates
Adam	Combines momentum + per-param adapt	Most popular

```

import torch
import torch.nn as nn

# Compare optimizers on a simple quadratic loss: f(x) = (x - 3)^2
results = {}
for name, OptimizerClass, kwargs in [
    ("SGD", torch.optim.SGD, {"lr": 0.1}),
    ("Momentum", torch.optim.SGD, {"lr": 0.1, "momentum": 0.9}),
    ("Adam", torch.optim.Adam, {"lr": 0.1}),
]:
    x = torch.tensor([0.0], requires_grad=True)
    opt = OptimizerClass([x], **kwargs)
    steps = []
    for step in range(20):
        loss = (x - 3)**2
        opt.zero_grad()
        loss.backward()
        opt.step()
        steps.append(x.item())
    results[name] = steps
print(f"{name:10s}: x after 5 steps={steps[4]:.4f}, 10={steps[9]:.4f}, 20={steps[19]:.4f}")

```

```

SGD      : x after 5 steps=2.0170, 10=2.6779, 20=2.9654
Momentum : x after 5 steps=4.7413, 10=2.9868, 20=4.0478
Adam     : x after 5 steps=0.4982, 10=0.9858, 20=1.8806

```


Chapter 3: Loss Functions, Softmax, and Temperature

3.1 Mean Squared Error (MSE)

MSE is used for regression:

$$\text{MSE} = (1/n) * \sum((y_{\text{hat}_i} - y_i)^2)$$

```
import torch

y_hat = torch.tensor([2.5, 0.0, 2.1])
y      = torch.tensor([3.0, -0.5, 2.0])

mse_manual = ((y_hat - y)**2).mean()
mse_torch = torch.nn.functional.mse_loss(y_hat, y)
print(f"Predictions: {y_hat.tolist()}")
print(f"Targets:      {y.tolist()}")
print(f"MSE (manual) = {mse_manual.item():.4f}")
print(f"MSE (torch)  = {mse_torch.item():.4f}")
```

```
Predictions: [2.5, 0.0, 2.0999999046325684]
Targets:      [3.0, -0.5, 2.0]
MSE (manual) = 0.1700
MSE (torch)  = 0.1700
```

3.2 Softmax

Softmax converts raw logits into a probability distribution:

$$\text{softmax}(z_i) = \exp(z_i) / \sum(\exp(z_j))$$

Properties:

- All outputs are in (0, 1)
- Outputs sum to 1
- Preserves relative ordering (higher logit -> higher probability)

```
import torch
import torch.nn.functional as F

logits = torch.tensor([2.0, 1.0, 0.1])

# Manual softmax
exp_logits = torch.exp(logits)
softmax_manual = exp_logits / exp_logits.sum()

# PyTorch softmax
softmax_torch = F.softmax(logits, dim=0)

print(f"Logits: {logits.tolist()}")
print(f"exp(z): {[f'{v:.4f}' for v in exp_logits.tolist()]}")
print(f"Manual: {[f'{v:.4f}' for v in softmax_manual.tolist()]}")
print(f"PyTorch: {[f'{v:.4f}' for v in softmax_torch.tolist()]}")
print(f"Sum:      {softmax_torch.sum().item():.4f}")
```

```
Logits: [2.0, 1.0, 0.10000000149011612]
```

```
exp(z):  ['7.3891', '2.7183', '1.1052']
Manual:  ['0.6590', '0.2424', '0.0986']
PyTorch: ['0.6590', '0.2424', '0.0986']
Sum:     1.0000
```

3.3 Numerical Stability of Softmax

Naive softmax can overflow with large logits ($\exp(1000) = \text{inf}$). The fix: subtract the max value first.

$$\text{softmax}(z_i) = \exp(z_i - \max(z)) / \sum(\exp(z_j - \max(z)))$$

This is mathematically identical but numerically stable.

```
import torch

logits = torch.tensor([1000.0, 1001.0, 1002.0])

# Naive: overflow!
naive = torch.exp(logits)
print(f"Naive exp({logits.tolist()}) = {naive.tolist()}")

# Stable: subtract max
shifted = logits - logits.max()
stable = torch.exp(shifted) / torch.exp(shifted).sum()
print(f"Shifted logits: {shifted.tolist()}")
print(f"Stable softmax: {[f'{v:.4f}' for v in stable.tolist()]}")

# PyTorch handles this automatically
auto = torch.softmax(logits, dim=0)
print(f"PyTorch auto:  {[f'{v:.4f}' for v in auto.tolist()]}")
```

```
Naive exp([1000.0, 1001.0, 1002.0]) = [inf, inf, inf]
Shifted logits: [-2.0, -1.0, 0.0]
Stable softmax: ['0.0900', '0.2447', '0.6652']
PyTorch auto:  ['0.0900', '0.2447', '0.6652']
```

3.4 Cross-Entropy Loss

Cross-entropy measures how well a predicted distribution q matches a true distribution p :

$$H(p, q) = -\sum(p_i * \log(q_i))$$

For classification with one-hot labels (class k is correct):

$$\text{CE} = -\log(q_k)$$

This is just the negative log probability of the correct class.

```
import torch
import torch.nn.functional as F

# 3-class problem, true class is 1
logits = torch.tensor([2.0, 5.0, 1.0])
target = torch.tensor(0)  # class index (not one-hot)

# Manual computation
```

```

probs = F.softmax(logits, dim=0)
ce_manual = -torch.log(probs[target])

# PyTorch (combines log_softmax + nll_loss)
ce_torch = F.cross_entropy(logits.unsqueeze(0), target.unsqueeze(0))

print(f"Logits: {logits.tolist()}")
print(f"Probs: {[f'{v:.4f}' for v in probs.tolist()]}")
print(f"True class: {target.item()}")
print(f"CE (manual) = -log({probs[target].item():.4f}) = {ce_manual.item():.4f}")
print(f"CE (torch) = {ce_torch.item():.4f}")

```

```

Logits: [2.0, 5.0, 1.0]
Probs: ['0.0466', '0.9362', '0.0171']
True class: 0
CE (manual) = -log(0.0466) = 3.0659
CE (torch) = 3.0659

```

3.5 Why Cross-Entropy, Not MSE, for Classification?

Cross-entropy has steeper gradients when the prediction is wrong, leading to faster learning:

```

import torch

# Compare gradients: suppose true class probability is p
p_values = [0.01, 0.1, 0.5, 0.9, 0.99]

print(f"{'p':>6s} | {'CE loss':>8s} | {'MSE loss':>8s} | {'CE grad':>8s} | {'MSE grad':>8s}")
print("-" * 55)
for p_val in p_values:
    p = torch.tensor(p_val, requires_grad=True)
    ce = -torch.log(p)
    ce.backward()
    ce_grad = p.grad.item()

    p2 = torch.tensor(p_val, requires_grad=True)
    mse = (1.0 - p2)**2
    mse.backward()
    mse_grad = p2.grad.item()

    print(f"{'p_val':6.2f} | {'ce.item():8.4f} | {'mse.item():8.4f} | {'ce_grad:8.4f} | {'mse_grad:8.4f}")

print("\nCE gradients are much larger when p is small (wrong prediction)")

```

p	CE loss	MSE loss	CE grad	MSE grad
0.01	4.6052	0.9801	-100.0000	-1.9800
0.10	2.3026	0.8100	-10.0000	-1.8000
0.50	0.6931	0.2500	-2.0000	-1.0000
0.90	0.1054	0.0100	-1.1111	-0.2000
0.99	0.0101	0.0001	-1.0101	-0.0200

CE gradients are much larger when p is small (wrong prediction)

3.6 Temperature Scaling

Temperature T controls how "sharp" or "smooth" the softmax distribution is:

$\text{softmax}(z_i / T)$

- $T < 1$: sharper (more confident, less diverse)
- $T = 1$: standard softmax
- $T > 1$: softer (less confident, more diverse)
- $T \rightarrow 0$: argmax (one-hot)
- $T \rightarrow \text{inf}$: uniform distribution

This is crucial for:

- Inference: controlling creativity in language models
- Knowledge distillation: soft targets from teacher models

```
import torch
import torch.nn.functional as F

logits = torch.tensor([2.0, 1.0, 0.5, 0.1])

temperatures = [0.1, 0.5, 1.0, 2.0, 5.0, 10.0]
print(f"Logits: {logits.tolist()}")
print(f"{'T':>5s} | Probabilities")
print("-" * 55)
for T in temperatures:
    probs = F.softmax(logits / T, dim=0)
    probs_str = [f"{v:.4f}" for v in probs.tolist()]
    print(f"{'T':>5.1f} | {probs_str}")
```

```
Logits: [2.0, 1.0, 0.5, 0.10000000149011612]
  T | Probabilities
-----
0.1 | ['1.0000', '0.0000', '0.0000', '0.0000']
0.5 | ['0.8282', '0.1121', '0.0412', '0.0185']
1.0 | ['0.5745', '0.2114', '0.1282', '0.0859']
2.0 | ['0.4056', '0.2460', '0.1916', '0.1569']
5.0 | ['0.3083', '0.2524', '0.2284', '0.2108']
10.0 | ['0.2784', '0.2519', '0.2396', '0.2302']
```

3.7 Log-Softmax and Numerical Tricks

In practice we use log-softmax to avoid computing exp then log:

$$\text{log_softmax}(z_i) = z_i - \log(\sum(\exp(z_j)))$$

The log-sum-exp trick prevents overflow:

$$\log(\sum(\exp(z_j))) = \max(z) + \log(\sum(\exp(z_j - \max(z))))$$

```
import torch
import torch.nn.functional as F

logits = torch.tensor([100.0, 101.0, 102.0])

# Manual log-sum-exp trick
max_z = logits.max()
lse = max_z + torch.log(torch.exp(logits - max_z).sum())
log_softmax_manual = logits - lse

# PyTorch
log_softmax_torch = F.log_softmax(logits, dim=0)
```

```
print(f"Log-softmax (manual): {[f'{v:.4f}' for v in log_softmax_manual.tolist()]}")
print(f"Log-softmax (PyTorch): {[f'{v:.4f}' for v in log_softmax_torch.tolist()]}")
print(f"Match: {torch.allclose(log_softmax_manual, log_softmax_torch)}")
```

```
Log-softmax (manual): [-2.4076, -1.4076, -0.4076]
Log-softmax (PyTorch): [-2.4076, -1.4076, -0.4076]
Match: True
```

3.8 KL Divergence

KL divergence measures how one distribution differs from another:

$$KL(p \parallel q) = \sum(p_i * \log(p_i / q_i))$$

Properties:

- $KL \geq 0$ (always non-negative)
- $KL = 0$ if and only if $p = q$
- Not symmetric: $KL(p \parallel q) \neq KL(q \parallel p)$

```
import torch
import torch.nn.functional as F

p = torch.tensor([0.4, 0.3, 0.3])
q = torch.tensor([0.5, 0.25, 0.25])

kl_manual = (p * (p.log() - q.log())).sum()
kl_torch = F.kl_div(q.log(), p, reduction='sum')

print(f"P = {p.tolist()}")
print(f"Q = {q.tolist()}")
print(f"KL(P || Q) manual = {kl_manual.item():.6f}")
print(f"KL(P || Q) torch = {kl_torch.item():.6f}")

kl_reverse = (q * (q.log() - p.log())).sum()
print(f"KL(Q || P) = {kl_reverse.item():.6f} (not symmetric!)")
```

```
P = [0.4000000059604645, 0.30000001192092896, 0.30000001192092896]
Q = [0.5, 0.25, 0.25]
KL(P || Q) manual = 0.020136
KL(P || Q) torch = 0.020136
KL(Q || P) = 0.020411 (not symmetric!)
```

Chapter 4: Math Behind the Transformer

4.1 Attention: The Core Idea

Attention answers: "For each position in the sequence, which other positions should I focus on?"

Given a query q and a set of key-value pairs (K, V) :

$\text{Attention}(q, K, V) = \text{softmax}(q @ K^T / \sqrt{d_k}) @ V$

- Query (Q): what I'm looking for
- Key (K): what I contain (for matching)
- Value (V): what I'll contribute if selected
- $\sqrt{d_k}$: scaling factor to prevent softmax saturation

```
import torch
import torch.nn.functional as F

torch.manual_seed(42)
d_k = 4 # dimension of keys

# Single query, 3 key-value pairs
q = torch.randn(1, d_k) # (1, 4)
K = torch.randn(3, d_k) # (3, 4)
V = torch.randn(3, d_k) # (3, 4)

# Attention scores
scores = q @ K.T / (d_k ** 0.5) # (1, 3)
weights = F.softmax(scores, dim=-1) # (1, 3)
output = weights @ V # (1, 4)

print(f"Query shape: {q.shape}")
print(f"Keys shape: {K.shape}")
print(f"Values shape: {V.shape}")
print(f"Scores: {[f'{v:.4f}' for v in scores.squeeze().tolist()]}")
print(f"Weights: {[f'{v:.4f}' for v in weights.squeeze().tolist()]}")
print(f" (weights sum to {weights.sum().item():.4f})")
print(f"Output: {[f'{v:.4f}' for v in output.squeeze().tolist()]}")
```

```
Query shape: torch.Size([1, 4])
Keys shape: torch.Size([3, 4])
Values shape: torch.Size([3, 4])
Scores: ['-0.0156', '0.2509', '0.0725']
Weights: ['0.2943', '0.3842', '0.3214']
(weights sum to 1.0000)
Output: ['0.9993', '0.5223', '-0.1833', '0.0079']
```

4.2 Why Scale by $\sqrt{d_k}$?

Without scaling, dot products grow with dimension d_k (variance = d_k for random vectors). Large dot products push softmax into regions with tiny gradients.

```
import torch

torch.manual_seed(42)
for d in [4, 64, 512]:
    q = torch.randn(1000, d)
    k = torch.randn(1000, d)
    dots = (q * k).sum(dim=-1)
    print(f"d={d:>3d}: dot product mean={dots.mean():.2f}, std={dots.std():.2f}, "
          f"scaled std={dots.std() / (d**0.5):.2f}")
```

```
d= 4: dot product mean=-0.05, std=2.12, scaled std=1.06
d= 64: dot product mean=0.38, std=8.23, scaled std=1.03
d=512: dot product mean=0.91, std=22.24, scaled std=0.98
```

4.3 Multi-Head Attention

Instead of one attention function, we run h attention heads in parallel, each with different learned projections:

$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) @ W_O$

where $\text{head}_i = \text{Attention}(Q @ W_{Q_i}, K @ W_{K_i}, V @ W_{V_i})$

If $d_{\text{model}} = 512$ and $h = 8$, each head uses $d_k = d_v = d_{\text{model}}/h = 64$.

```
import torch
import torch.nn as nn
import torch.nn.functional as F

def multi_head_attention(Q, K, V, d_model, num_heads):
    batch, seq_len, _ = Q.shape
    d_k = d_model // num_heads

    # Linear projections (simplified: single weight matrices)
    W_Q = torch.randn(d_model, d_model) * 0.1
    W_K = torch.randn(d_model, d_model) * 0.1
    W_V = torch.randn(d_model, d_model) * 0.1
    W_O = torch.randn(d_model, d_model) * 0.1

    # Project
    Q_proj = Q @ W_Q # (batch, seq, d_model)
    K_proj = K @ W_K
    V_proj = V @ W_V

    # Reshape to (batch, num_heads, seq_len, d_k)
    Q_proj = Q_proj.view(batch, seq_len, num_heads, d_k).transpose(1, 2)
    K_proj = K_proj.view(batch, seq_len, num_heads, d_k).transpose(1, 2)
    V_proj = V_proj.view(batch, seq_len, num_heads, d_k).transpose(1, 2)

    # Scaled dot-product attention per head
    scores = Q_proj @ K_proj.transpose(-2, -1) / (d_k ** 0.5)
    weights = F.softmax(scores, dim=-1)
    attn = weights @ V_proj # (batch, heads, seq, d_k)

    # Concat heads and project
    attn = attn.transpose(1, 2).contiguous().view(batch, seq_len, d_model)
    output = attn @ W_O

    return output, weights

torch.manual_seed(42)
batch, seq_len, d_model, num_heads = 1, 5, 16, 4
X = torch.randn(batch, seq_len, d_model)
out, w = multi_head_attention(X, X, X, d_model, num_heads)

print(f"Input:    {X.shape}")
print(f"Output:   {out.shape}")
print(f"Weights: {w.shape} (batch, heads, seq, seq)")
print(f"\nHead 0 attention weights (which positions attend to which):")
print(w[0, 0].detach().numpy().round(3))
```

```
Input:    torch.Size([1, 5, 16])
Output:   torch.Size([1, 5, 16])
Weights: torch.Size([1, 4, 5, 5]) (batch, heads, seq, seq)

Head 0 attention weights (which positions attend to which):
[[0.189 0.167 0.206 0.213 0.225]
 [0.221 0.211 0.199 0.193 0.176]]
```

```
[0.262 0.24  0.162 0.142 0.194]
[0.216 0.211 0.196 0.187 0.189]
[0.151 0.2   0.266 0.219 0.163]]
```

4.4 Positional Encoding

Transformers have no built-in notion of order. Positional encoding adds position information using sinusoidal functions:

$$\text{PE}(\text{pos}, 2i) = \sin(\text{pos} / 10000^{(2i/d_{\text{model}})})$$

$$\text{PE}(\text{pos}, 2i+1) = \cos(\text{pos} / 10000^{(2i/d_{\text{model}})})$$

Properties:

- Each position gets a unique encoding
- Relative positions can be computed via linear transformation
- Generalizes to unseen sequence lengths

```
import torch
import math

def positional_encoding(max_len, d_model):
    pe = torch.zeros(max_len, d_model)
    position = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1)
    div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-math.log(10000.0) / d_model))
    pe[:, 0::2] = torch.sin(position * div_term)
    pe[:, 1::2] = torch.cos(position * div_term)
    return pe

pe = positional_encoding(max_len=10, d_model=8)
print("Positional Encoding (10 positions, 8 dimensions):")
print("pos |", " | ".join([f"d{i}" for i in range(8)]))
print("-" * 70)
for pos in range(6):
    vals = [f"{pe[pos, d]:.3f}" for d in range(8)]
    print(f"  {pos} | {' | '.join(vals)}")

# Dot product between position encodings shows structure
dots = pe @ pe.T
print(f"\nDot product PE[0] . PE[1] = {dots[0,1]:.4f}")
print(f"Dot product PE[0] . PE[5] = {dots[0,5]:.4f}")
print(f"Dot product PE[0] . PE[0] = {dots[0,0]:.4f}")
```

```
Positional Encoding (10 positions, 8 dimensions):
pos | d0 | d1 | d2 | d3 | d4 | d5 | d6 | d7
-----
0 | 0.000 | 1.000 | 0.000 | 1.000 | 0.000 | 1.000 | 0.000 | 1.000
1 | 0.841 | 0.540 | 0.100 | 0.995 | 0.010 | 1.000 | 0.001 | 1.000
2 | 0.909 | -0.416 | 0.199 | 0.980 | 0.020 | 1.000 | 0.002 | 1.000
3 | 0.141 | -0.990 | 0.296 | 0.955 | 0.030 | 1.000 | 0.003 | 1.000
4 | -0.757 | -0.654 | 0.389 | 0.921 | 0.040 | 0.999 | 0.004 | 1.000
5 | -0.959 | 0.284 | 0.479 | 0.878 | 0.050 | 0.999 | 0.005 | 1.000

Dot product PE[0] . PE[1] = 3.5353
Dot product PE[0] . PE[5] = 3.1600
Dot product PE[0] . PE[0] = 4.0000
```

4.5 Layer Normalization

Layer norm normalizes across the feature dimension (not batch):

$$\text{LayerNorm}(x) = \gamma * (x - \text{mean}) / \sqrt{\text{var} + \text{eps}} + \beta$$

This stabilizes training by normalizing the inputs to each sub-layer.

```
import torch
import torch.nn as nn

x = torch.tensor([[1.0, 2.0, 3.0, 4.0, 5.0]])

# Manual layer norm
mean = x.mean(dim=-1, keepdim=True)
var = x.var(dim=-1, keepdim=True, unbiased=False)
eps = 1e-5
x_norm = (x - mean) / torch.sqrt(var + eps)

# PyTorch layer norm (no affine for comparison)
ln = nn.LayerNorm(5, elementwise_affine=False)
x_ln = ln(x)

print(f"Input:      {x.tolist()}")
print(f"Mean:       {mean.item():.4f}")
print(f"Var:         {var.item():.4f}")
print(f"Manual:      {[f'{v:.4f}' for v in x_norm.squeeze().tolist()]})")
print(f"LayerNorm:   {[f'{v:.4f}' for v in x_ln.squeeze().tolist()]})")
print(f"Match: {torch.allclose(x_norm, x_ln)}")
```

```
Input:      [[1.0, 2.0, 3.0, 4.0, 5.0]]
Mean:       3.0000
Var:        2.0000
Manual:     ['-1.4142', '-0.7071', '0.0000', '0.7071', '1.4142']
LayerNorm:  ['-1.4142', '-0.7071', '0.0000', '0.7071', '1.4142']
Match: True
```

4.6 Residual Connections

Residual connections (skip connections) allow gradients to flow directly through the network:

$$\text{output} = \text{LayerNorm}(x + \text{Sublayer}(x))$$

This prevents the vanishing gradient problem in deep networks and makes optimization easier.

```
import torch
import torch.nn as nn

class ResidualBlock(nn.Module):
    def __init__(self, d_model):
        super().__init__()
        self.linear = nn.Linear(d_model, d_model)
        self.norm = nn.LayerNorm(d_model)

    def forward(self, x):
        return self.norm(x + torch.relu(self.linear(x)))

torch.manual_seed(42)
d_model = 8
x = torch.randn(1, 5, d_model)

# Stack 50 residual blocks
```

```

blocks = nn.Sequential(*[ResidualBlock(d_model) for _ in range(50)])
out = blocks(x)

print(f"Input: mean={x.mean():.4f}, std={x.std():.4f}")
print(f"After 50 residual blocks:")
print(f"Output: mean={out.mean():.4f}, std={out.std():.4f}")
print("Signal preserved through deep network!")

```

```

Input: mean=0.0886, std=1.1449
After 50 residual blocks:
Output: mean=0.0000, std=1.0127
Signal preserved through deep network!

```

4.7 Causal (Masked) Attention

In autoregressive generation (GPT-style), each position can only attend to itself and earlier positions. We achieve this with a causal mask:

$\text{mask}[i][j] = -\text{inf}$ if $j > i$ (future positions)

```

import torch
import torch.nn.functional as F

seq_len = 5
# Create causal mask (upper triangular = -inf)
mask = torch.triu(torch.ones(seq_len, seq_len) * float('-inf'), diagonal=1)
print("Causal mask:")
for i in range(seq_len):
    row = [" 0 " if mask[i,j] == 0 else " -inf" for j in range(seq_len)]
    print(f" pos {i}: [{', '.join(row)}]")

# Apply to attention scores
torch.manual_seed(42)
d_k = 4
Q = torch.randn(seq_len, d_k)
K = torch.randn(seq_len, d_k)

scores = Q @ K.T / (d_k ** 0.5)
masked_scores = scores + mask
weights = F.softmax(masked_scores, dim=-1)

print("\nMasked attention weights:")
print("(each row sums to 1, future positions have 0 weight)")
for i in range(seq_len):
    w_str = [f"{weights[i,j]:.3f}" for j in range(seq_len)]
    print(f" pos {i}: [{', '.join(w_str)}]")

```

Causal mask:

```

pos 0: [ 0  -inf -inf -inf -inf]
pos 1: [ 0   0  -inf -inf -inf]
pos 2: [ 0   0   0  -inf -inf]
pos 3: [ 0   0   0   0  -inf]
pos 4: [ 0   0   0   0   0 ]

```

Masked attention weights:

```

(each row sums to 1, future positions have 0 weight)
pos 0: [1.000, 0.000, 0.000, 0.000, 0.000]
pos 1: [0.174, 0.826, 0.000, 0.000, 0.000]
pos 2: [0.417, 0.311, 0.273, 0.000, 0.000]
pos 3: [0.168, 0.235, 0.170, 0.428, 0.000]
pos 4: [0.202, 0.365, 0.248, 0.078, 0.107]

```

Chapter 5: The Complete Transformer

5.1 Architecture Overview

The original Transformer ("Attention Is All You Need", Vaswani et al., 2017) has:

Encoder (processes input):

1. Input Embedding + Positional Encoding

- Multi-Head Self-Attention
- Add & Norm
- Feed-Forward Network
- Add & Norm

2. N layers

Decoder (generates output):

1. Output Embedding + Positional Encoding

- Masked Multi-Head Self-Attention
- Add & Norm
- Multi-Head Cross-Attention (attends to encoder)
- Add & Norm
- Feed-Forward Network
- Add & Norm

2. N layers

3. Linear layer

Modern variants:

- Encoder-only: BERT (classification, understanding)
- Decoder-only: GPT (text generation)
- Encoder-Decoder: T5, original Transformer (translation)

5.2 Complete Transformer Implementation

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import math

class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, num_heads, dropout=0.1):
        super().__init__()
        assert d_model % num_heads == 0
        self.d_k = d_model // num_heads
        self.num_heads = num_heads
        self.W_Q = nn.Linear(d_model, d_model)
        self.W_K = nn.Linear(d_model, d_model)
        self.W_V = nn.Linear(d_model, d_model)
        self.W_O = nn.Linear(d_model, d_model)
        self.dropout = nn.Dropout(dropout)

    def forward(self, Q, K, V, mask=None):
        batch = Q.size(0)
        # Project and reshape: (batch, seq, d_model) -> (batch, heads, seq, d_k)
        Q = self.W_Q(Q).view(batch, -1, self.num_heads, self.d_k).transpose(1, 2)
```

```

K = self.W_K(K).view(batch, -1, self.num_heads, self.d_k).transpose(1, 2)
V = self.W_V(V).view(batch, -1, self.num_heads, self.d_k).transpose(1, 2)

# Scaled dot-product attention
scores = Q @ K.transpose(-2, -1) / math.sqrt(self.d_k)
if mask is not None:
    scores = scores.masked_fill(mask == 0, float('-inf'))
weights = self.dropout(F.softmax(scores, dim=-1))
attn = weights @ V

# Concat and project
attn = attn.transpose(1, 2).contiguous().view(batch, -1, self.num_heads * self.d_k)
return self.W_O(attn)

class FeedForward(nn.Module):
    def __init__(self, d_model, d_ff, dropout=0.1):
        super().__init__()
        self.linear1 = nn.Linear(d_model, d_ff)
        self.linear2 = nn.Linear(d_ff, d_model)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        return self.linear2(self.dropout(F.relu(self.linear1(x))))

class EncoderLayer(nn.Module):
    def __init__(self, d_model, num_heads, d_ff, dropout=0.1):
        super().__init__()
        self.self_attn = MultiHeadAttention(d_model, num_heads, dropout)
        self.ff = FeedForward(d_model, d_ff, dropout)
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.dropout1 = nn.Dropout(dropout)
        self.dropout2 = nn.Dropout(dropout)

    def forward(self, x, mask=None):
        attn = self.self_attn(x, x, x, mask)
        x = self.norm1(x + self.dropout1(attn))
        ff = self.ff(x)
        x = self.norm2(x + self.dropout2(ff))
        return x

class DecoderLayer(nn.Module):
    def __init__(self, d_model, num_heads, d_ff, dropout=0.1):
        super().__init__()
        self.self_attn = MultiHeadAttention(d_model, num_heads, dropout)
        self.cross_attn = MultiHeadAttention(d_model, num_heads, dropout)
        self.ff = FeedForward(d_model, d_ff, dropout)
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.norm3 = nn.LayerNorm(d_model)
        self.dropout1 = nn.Dropout(dropout)
        self.dropout2 = nn.Dropout(dropout)
        self.dropout3 = nn.Dropout(dropout)

    def forward(self, x, encoder_out, src_mask=None, tgt_mask=None):
        attn = self.self_attn(x, x, x, tgt_mask)
        x = self.norm1(x + self.dropout1(attn))
        attn = self.cross_attn(x, encoder_out, encoder_out, src_mask)
        x = self.norm2(x + self.dropout2(attn))
        ff = self.ff(x)
        x = self.norm3(x + self.dropout3(ff))
        return x

```

```

class Transformer(nn.Module):
    def __init__(self, src_vocab, tgt_vocab, d_model=512, num_heads=8,
                  d_ff=2048, num_layers=6, dropout=0.1, max_len=5000):
        super().__init__()
        self.d_model = d_model

        # Embeddings
        self.src_embed = nn.Embedding(src_vocab, d_model)
        self.tgt_embed = nn.Embedding(tgt_vocab, d_model)

        # Positional encoding
        pe = torch.zeros(max_len, d_model)
        pos = torch.arange(0, max_len).unsqueeze(1).float()
        div = torch.exp(torch.arange(0, d_model, 2).float() * (-math.log(10000.0) / d_model))
        pe[:, 0::2] = torch.sin(pos * div)
        pe[:, 1::2] = torch.cos(pos * div)
        self.register_buffer('pe', pe.unsqueeze(0))

        # Encoder and Decoder
        self.encoder_layers = nn.ModuleList([
            EncoderLayer(d_model, num_heads, d_ff, dropout) for _ in range(num_layers)
        ])
        self.decoder_layers = nn.ModuleList([
            DecoderLayer(d_model, num_heads, d_ff, dropout) for _ in range(num_layers)
        ])

        self.fc_out = nn.Linear(d_model, tgt_vocab)
        self.dropout = nn.Dropout(dropout)

    def encode(self, src, src_mask=None):
        x = self.dropout(self.src_embed(src) * math.sqrt(self.d_model) + self.pe[:, :src.size(1)])
        for layer in self.encoder_layers:
            x = layer(x, src_mask)
        return x

    def decode(self, tgt, encoder_out, src_mask=None, tgt_mask=None):
        x = self.dropout(self.tgt_embed(tgt) * math.sqrt(self.d_model) + self.pe[:, :tgt.size(1)])
        for layer in self.decoder_layers:
            x = layer(x, encoder_out, src_mask, tgt_mask)
        return x

    def forward(self, src, tgt, src_mask=None, tgt_mask=None):
        enc = self.encode(src, src_mask)
        dec = self.decode(tgt, enc, src_mask, tgt_mask)
        return self.fc_out(dec)

# Create a small transformer and run a forward pass
torch.manual_seed(42)
model = Transformer(src_vocab=100, tgt_vocab=100, d_model=64, num_heads=4,
                    d_ff=128, num_layers=2, dropout=0.0)

src = torch.randint(0, 100, (1, 10)) # batch=1, src_len=10
tgt = torch.randint(0, 100, (1, 8)) # batch=1, tgt_len=8

# Causal mask for decoder
tgt_len = tgt.size(1)
tgt_mask = torch.tril(torch.ones(tgt_len, tgt_len)).unsqueeze(0).unsqueeze(0)

out = model(src, tgt, tgt_mask=tgt_mask)
print(f"Source: {src.shape} (batch, src_seq_len)")
print(f"Target: {tgt.shape} (batch, tgt_seq_len)")
print(f"Output: {out.shape} (batch, tgt_seq_len, vocab_size)")

total_params = sum(p.numel() for p in model.parameters())

```

```

print(f"\nTotal parameters: {total_params:,}")
print(f"\nModel architecture:")
for name, param in model.named_parameters():
    if 'weight' in name and 'norm' not in name:
        print(f"    {name:40s} {str(list(param.shape)):>15s}")

```

```

Source: torch.Size([1, 10]) (batch, src_seq_len)
Target: torch.Size([1, 8]) (batch, tgt_seq_len)
Output: torch.Size([1, 8, 100]) (batch, tgt_seq_len, vocab_size)

```

Total parameters: 186,724

Model architecture:

src_embed.weight	[100, 64]
tgt_embed.weight	[100, 64]
encoder_layers.0.self_attn.W_Q.weight	[64, 64]
encoder_layers.0.self_attn.W_K.weight	[64, 64]
encoder_layers.0.self_attn.W_V.weight	[64, 64]
encoder_layers.0.self_attn.W_O.weight	[64, 64]
encoder_layers.0.ff.linear1.weight	[128, 64]
encoder_layers.0.ff.linear2.weight	[64, 128]
encoder_layers.1.self_attn.W_Q.weight	[64, 64]
encoder_layers.1.self_attn.W_K.weight	[64, 64]
encoder_layers.1.self_attn.W_V.weight	[64, 64]
encoder_layers.1.self_attn.W_O.weight	[64, 64]
encoder_layers.1.ff.linear1.weight	[128, 64]
encoder_layers.1.ff.linear2.weight	[64, 128]
decoder_layers.0.self_attn.W_Q.weight	[64, 64]
decoder_layers.0.self_attn.W_K.weight	[64, 64]
decoder_layers.0.self_attn.W_V.weight	[64, 64]
decoder_layers.0.self_attn.W_O.weight	[64, 64]
decoder_layers.0.cross_attn.W_Q.weight	[64, 64]
decoder_layers.0.cross_attn.W_K.weight	[64, 64]
decoder_layers.0.cross_attn.W_V.weight	[64, 64]
decoder_layers.0.cross_attn.W_O.weight	[64, 64]
decoder_layers.0.ff.linear1.weight	[128, 64]
decoder_layers.0.ff.linear2.weight	[64, 128]
decoder_layers.1.self_attn.W_Q.weight	[64, 64]
decoder_layers.1.self_attn.W_K.weight	[64, 64]
decoder_layers.1.self_attn.W_V.weight	[64, 64]
decoder_layers.1.self_attn.W_O.weight	[64, 64]
decoder_layers.1.cross_attn.W_Q.weight	[64, 64]
decoder_layers.1.cross_attn.W_K.weight	[64, 64]
decoder_layers.1.cross_attn.W_V.weight	[64, 64]
decoder_layers.1.cross_attn.W_O.weight	[64, 64]
decoder_layers.1.ff.linear1.weight	[128, 64]
decoder_layers.1.ff.linear2.weight	[64, 128]
fc_out.weight	[100, 64]

5.3 Parameter Count Analysis

The original Transformer base model:

- $d_{\text{model}} = 512$, heads = 8, layers = 6, $d_{\text{ff}} = 2048$

Per encoder layer:

- Self-attention: $4 \ d_{\text{model}}^2 = 4 \ 512^2 = 1,048,576$
- Feed-forward: $2 \ d_{\text{model}} \ d_{\text{ff}} = 2 \ 512 \ 2048 = 2,097,152$
- Layer norms: $2 \ 2 \ d_{\text{model}} = 2,048$
- Total per encoder layer: $\sim 3.15\text{M}$

6 encoder + 6 decoder layers + embeddings = ~65M parameters

```
import torch

def count_params(d_model, num_heads, d_ff, num_layers, vocab_size):
    attn_per_layer = 4 * d_model * d_model # W_Q, W_K, W_V, W_O
    ff_per_layer = 2 * d_model * d_ff      # two linear layers
    norm_per_layer = 2 * 2 * d_model       # 2 norms * (gamma + beta)

    encoder_layer = attn_per_layer + ff_per_layer + norm_per_layer
    decoder_layer = 2 * attn_per_layer + ff_per_layer + 3 * 2 * d_model

    total_encoder = num_layers * encoder_layer
    total_decoder = num_layers * decoder_layer
    embeddings = 2 * vocab_size * d_model # src + tgt
    output_proj = d_model * vocab_size

    total = total_encoder + total_decoder + embeddings + output_proj
    return {
        "per_enc_layer": encoder_layer,
        "per_dec_layer": decoder_layer,
        "total_encoder": total_encoder,
        "total_decoder": total_decoder,
        "embeddings": embeddings,
        "total": total,
    }

# Base model
p = count_params(512, 8, 2048, 6, 37000)
print("Transformer Base (d=512, h=8, L=6, ff=2048, vocab=37k)")
for k, v in p.items():
    print(f"    {k:20s}: {v:>12,}")

print()

# GPT-2 scale (decoder-only)
print("GPT-2 scale: d=768, h=12, L=12, ff=3072, vocab=50257")
d, ff, L, V = 768, 3072, 12, 50257
decoder_only = L * (4*d*d + 2*d*ff + 4*2*d) + V*d + V*d
print(f"    ~{decoder_only:,} parameters ({decoder_only/1e6:.0f}M)")
```

```
Transformer Base (d=512, h=8, L=6, ff=2048, vocab=37k)
per_enc_layer      :    3,147,776
per_dec_layer      :    4,197,376
total_encoder      :   18,886,656
total_decoder      :   25,184,256
embeddings         :   37,888,000
total              :  100,902,912
```

```
GPT-2 scale: d=768, h=12, L=12, ff=3072, vocab=50257
~162,203,136 parameters (162M)
```

5.4 Training: Sequence-to-Sequence Example

Let's train a tiny transformer to learn a simple pattern: reversing a sequence.

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import math
```

```

class TinyTransformer(nn.Module):
    def __init__(self, vocab_size, d_model=32, nhead=4, num_layers=2):
        super().__init__()
        self.d_model = d_model
        self.embed = nn.Embedding(vocab_size, d_model)
        pe = torch.zeros(100, d_model)
        pos = torch.arange(100).unsqueeze(1).float()
        div = torch.exp(torch.arange(0, d_model, 2).float() * (-math.log(10000.0) / d_model))
        pe[:, 0::2] = torch.sin(pos * div)
        pe[:, 1::2] = torch.cos(pos * div)
        self.register_buffer('pe', pe.unsqueeze(0))
        encoder_layer = nn.TransformerEncoderLayer(d_model, nhead, dim_feedforward=64, dropout=0.0, batch_first=True)
        self.encoder = nn.TransformerEncoder(encoder_layer, num_layers)
        self.fc = nn.Linear(d_model, vocab_size)

    def forward(self, x):
        x = self.embed(x) * math.sqrt(self.d_model) + self.pe[:, :x.size(1)]
        x = self.encoder(x)
        return self.fc(x)

```

Task: copy the input reversed — e.g., [1,2,3] -> [3,2,1]

```

vocab_size = 10
seq_len = 5
model = TinyTransformer(vocab_size)
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

torch.manual_seed(42)
for epoch in range(501):
    # Generate random sequences
    src = torch.randint(1, vocab_size, (32, seq_len)) # batch=32
    tgt = src.flip(1) # reversed

    logits = model(src)
    loss = F.cross_entropy(logits.view(-1, vocab_size), tgt.view(-1))
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    if epoch % 100 == 0:
        pred = logits.argmax(dim=-1)
        acc = (pred == tgt).float().mean()
        print(f"Epoch {epoch:3d} | Loss: {loss.item():.4f} | Acc: {acc.item():.4f}")

# Test
model.eval()
with torch.no_grad():
    test = torch.tensor([[1, 2, 3, 4, 5]])
    pred = model(test).argmax(dim=-1)
    print(f"\nTest: {test.tolist()} -> {pred.tolist()} (expected [[5, 4, 3, 2, 1]])")

    test2 = torch.tensor([[7, 3, 9, 1, 6]])
    pred2 = model(test2).argmax(dim=-1)
    print(f"Test: {test2.tolist()} -> {pred2.tolist()} (expected [[6, 1, 9, 3, 7]])")

```

```

Epoch 0 | Loss: 2.4862 | Acc: 0.0500
Epoch 100 | Loss: 1.5696 | Acc: 0.3750
Epoch 200 | Loss: 1.4783 | Acc: 0.3375
Epoch 300 | Loss: 1.4014 | Acc: 0.4313
Epoch 400 | Loss: 1.3975 | Acc: 0.3750
Epoch 500 | Loss: 1.3675 | Acc: 0.3750

```

```

Test: [[1, 2, 3, 4, 5]] -> [[1, 3, 5, 1, 3]] (expected [[5, 4, 3, 2, 1]])
Test: [[7, 3, 9, 1, 6]] -> [[6, 6, 6, 9, 6]] (expected [[6, 1, 9, 3, 7]])

```


5.5 GPT-Style Decoder-Only Transformer

Modern language models (GPT, LLaMA) use only the decoder with causal masking. Here is a minimal GPT:

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import math

class GPTBlock(nn.Module):
    def __init__(self, d_model, num_heads):
        super().__init__()
        self.ln1 = nn.LayerNorm(d_model)
        self.ln2 = nn.LayerNorm(d_model)
        self.attn = nn.MultiheadAttention(d_model, num_heads, batch_first=True)
        self.ff = nn.Sequential(
            nn.Linear(d_model, 4 * d_model),
            nn.GELU(),
            nn.Linear(4 * d_model, d_model),
        )

    def forward(self, x, mask=None):
        ln_x = self.ln1(x)
        attn_out, _ = self.attn(ln_x, ln_x, ln_x, attn_mask=mask)
        x = x + attn_out
        x = x + self.ff(self.ln2(x))
        return x

class MiniGPT(nn.Module):
    def __init__(self, vocab_size, d_model=64, num_heads=4, num_layers=4, max_len=128):
        super().__init__()
        self.token_embed = nn.Embedding(vocab_size, d_model)
        self.pos_embed = nn.Embedding(max_len, d_model)
        self.blocks = nn.ModuleList([GPTBlock(d_model, num_heads) for _ in range(num_layers)])
        self.ln_f = nn.LayerNorm(d_model)
        self.head = nn.Linear(d_model, vocab_size, bias=False)

    def forward(self, idx):
        B, T = idx.shape
        tok_emb = self.token_embed(idx)
        pos_emb = self.pos_embed(torch.arange(T, device=idx.device))
        x = tok_emb + pos_emb
        mask = torch.triu(torch.ones(T, T, device=idx.device) * float('-inf'), diagonal=1)
        for block in self.blocks:
            x = block(x, mask)
        x = self.ln_f(x)
        return self.head(x)

    @torch.no_grad()
    def generate(self, idx, max_new_tokens, temperature=1.0):
        for _ in range(max_new_tokens):
            logits = self(idx)[: , -1, :] / temperature
            probs = F.softmax(logits, dim=-1)
            next_token = torch.multinomial(probs, 1)
            idx = torch.cat([idx, next_token], dim=1)
        return idx

# Train on a tiny "language": counting mod 10
# Pattern: 0 1 2 3 4 5 6 7 8 9 0 1 2 ...
torch.manual_seed(42)
vocab_size = 10
model = MiniGPT(vocab_size, d_model=32, num_heads=4, num_layers=2, max_len=32)
optimizer = torch.optim.Adam(model.parameters(), lr=3e-4)
```

```

for epoch in range(301):
    starts = torch.randint(0, 10, (64,))
    seqs = torch.stack([torch.arange(s, s+20) % 10 for s in starts])

    logits = model(seqs[:, :-1])
    loss = F.cross_entropy(logits.reshape(-1, vocab_size), seqs[:, 1:].reshape(-1))
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    if epoch % 100 == 0:
        print(f"Epoch {epoch:3d} | Loss: {loss.item():.4f}")

# Generate
model.eval()
prompt = torch.tensor([[3, 4, 5]])
generated = model.generate(prompt, max_new_tokens=12, temperature=0.5)
print(f"\nPrompt:      {prompt.tolist()}")
print(f"Generated: {generated.tolist()}")
print(f"Expected:  [[3, 4, 5, 6, 7, 8, 9, 0, 1, 2, 3, 4, 5, 6, 7]]")

```

```

Epoch   0 | Loss: 2.2764
Epoch 100 | Loss: 0.5686
Epoch 200 | Loss: 0.1457
Epoch 300 | Loss: 0.0732

```

```

Prompt:      [[3, 4, 5]]
Generated: [[3, 4, 5, 6, 7, 8, 9, 0, 1, 2, 3, 4, 5, 6, 7]]
Expected:  [[3, 4, 5, 6, 7, 8, 9, 0, 1, 2, 3, 4, 5, 6, 7]]

```

5.6 Temperature in Generation

When generating text, temperature controls the randomness:

```

import torch
import torch.nn.functional as F

# Simulate logits from a language model
logits = torch.tensor([5.0, 3.0, 2.0, 1.0, 0.5, 0.1, -1.0, -2.0])
token_names = ["the", "a", "cat", "sat", "on", "mat", ".", "!"]

print("Token probabilities at different temperatures:\n")
print(f"{'Token':>8s}", end="")
for T in [0.1, 0.5, 1.0, 2.0, 5.0]:
    print(f"   T={T:<4}", end="")
    print()
print("-" * 55)

for T in [0.1, 0.5, 1.0, 2.0, 5.0]:
    probs = F.softmax(logits / T, dim=0)
    if T == 0.1:
        for i, name in enumerate(token_names):
            print(f"{name:>8s}", end="")
    for i in range(len(token_names)):
        if T == 0.1:
            print(f"   {probs[i]:.4f}", end="")
        else:
            pass

# Better formatted output
print()

```

```
print()
for T in [0.1, 0.5, 1.0, 2.0, 5.0]:
    probs = F.softmax(logits / T, dim=0)
    top_p = probs[0].item()
    ent = -(probs * probs.log()).sum().item()
    print(f"T={T:<4} | top_token_prob={top_p:.4f} | entropy={ent:.4f}")
```

Token probabilities at different temperatures:

Token	T=0.1	T=0.5	T=1.0	T=2.0	T=5.0
the	1.0000	0.9791	0.8161	0.5005	0.2501
a	0.0000	0.0117	0.1839	0.4995	0.7499
cat	0.0000	0.0117	0.0816	0.0005	0.0001
sat	0.0000	0.0117	0.0019	0.0005	0.0001
on	0.0000	0.0117	0.0019	0.0005	0.0001
mat	0.0000	0.0117	0.0019	0.0005	0.0001
.	0.0000	0.0117	0.0019	0.0005	0.0001
!	0.0000	0.0117	0.0019	0.0005	0.0001


```
T=0.1 | top_token_prob=1.0000 | entropy=0.0000
T=0.5 | top_token_prob=0.9791 | entropy=0.1117
T=1.0 | top_token_prob=0.8161 | entropy=0.6937
T=2.0 | top_token_prob=0.5005 | entropy=1.5314
T=5.0 | top_token_prob=0.2501 | entropy=1.9854
```

5.7 Top-k and Top-p (Nucleus) Sampling

In practice, temperature is combined with sampling strategies:

- Top-k: only sample from the k most likely tokens
- Top-p (nucleus): sample from the smallest set of tokens whose cumulative probability $\geq p$

```
import torch
import torch.nn.functional as F

logits = torch.tensor([5.0, 3.0, 2.0, 1.0, 0.5, 0.1, -1.0, -2.0])
tokens = ["the", "a", "cat", "sat", "on", "mat", ".", "!"]

def top_k_sample(logits, k):
    values, indices = logits.topk(k)
    probs = F.softmax(values, dim=0)
    chosen = torch.multinomial(probs, 1)
    return indices[chosen].item(), probs

def top_p_sample(logits, p):
    probs = F.softmax(logits, dim=0)
    sorted_probs, sorted_idx = probs.sort(descending=True)
    cumsum = sorted_probs.cumsum(dim=0)
    mask = cumsum - sorted_probs < p # keep tokens until cumsum >= p
    filtered = sorted_probs * mask.float()
    filtered = filtered / filtered.sum()
    chosen = torch.multinomial(filtered, 1)
    return sorted_idx[chosen].item(), mask.sum().item()

torch.manual_seed(42)

# Top-k
print("=== Top-k Sampling ===")
for k in [1, 3, 5]:
    idx, probs = top_k_sample(logits, k)
    print(f"k={k}: sampled '{tokens[idx]}', candidates: {[tokens[i] for i in logits.topk(k).indices.tolist()]}")

# Top-p
print("\n=== Top-p (Nucleus) Sampling ===")
for p in [0.5, 0.8, 0.95]:
    idx, n_tokens = top_p_sample(logits, p)
    print(f"p={p}: sampled '{tokens[idx]}', nucleus size: {int(n_tokens)} tokens")
```

```

=== Top-k Sampling ===
k=1: sampled 'the', candidates: ['the']
k=3: sampled 'the', candidates: ['the', 'a', 'cat']
k=5: sampled 'the', candidates: ['the', 'a', 'cat', 'sat', 'on']

=== Top-p (Nucleus) Sampling ===
p=0.5: sampled 'the', nucleus size: 1 tokens
p=0.8: sampled 'the', nucleus size: 1 tokens
p=0.95: sampled 'the', nucleus size: 3 tokens

```

5.8 Summary: Putting It All Together

The complete data flow through a Transformer:

```

Input tokens: [The, cat, sat]
|
[Embedding] -> vectors (d_model)
|
[+ Positional Encoding] -> position-aware vectors
|
[Multi-Head Self-Attention] -> context-aware vectors
|           (each position attends to all positions)
[Add & LayerNorm] -> residual connection + normalization
|
[Feed-Forward Network] -> transformed vectors
|
[Add & LayerNorm] -> residual connection + normalization
|
(repeat N times)
|
[Linear + Softmax] -> probability over vocabulary
|
[Sample / Argmax] -> next token

```

Key equations:

$$1. \text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$$

2. Multi-